In a type, && does not mean “and”.

It is a new kind of reference.

```
int&  a = x;
int&& b = 5;
```

The second one is an **rvalue reference**.

Added in C++11. It is what makes move semantics possible.

swipe →

Two kinds of value

	LVALUE	RVALUE
It is	a named object	a temporary
Lifetime	lives on after the line	gone at the `;`

Examples	`x`
	`s`
		`5`
		`a + b`
		`std::string("hi")`

Nobody can use an rvalue after this line. **So nobody will miss what is inside it.**

Each reference picks its own

TYPE	BINDS TO
`T&`	lvalues only
`const T&`	both (so it can copy)
`T&&`	**rvalues only**

```
void f(std::string&);    // 1
void f(std::string&&);   // 2

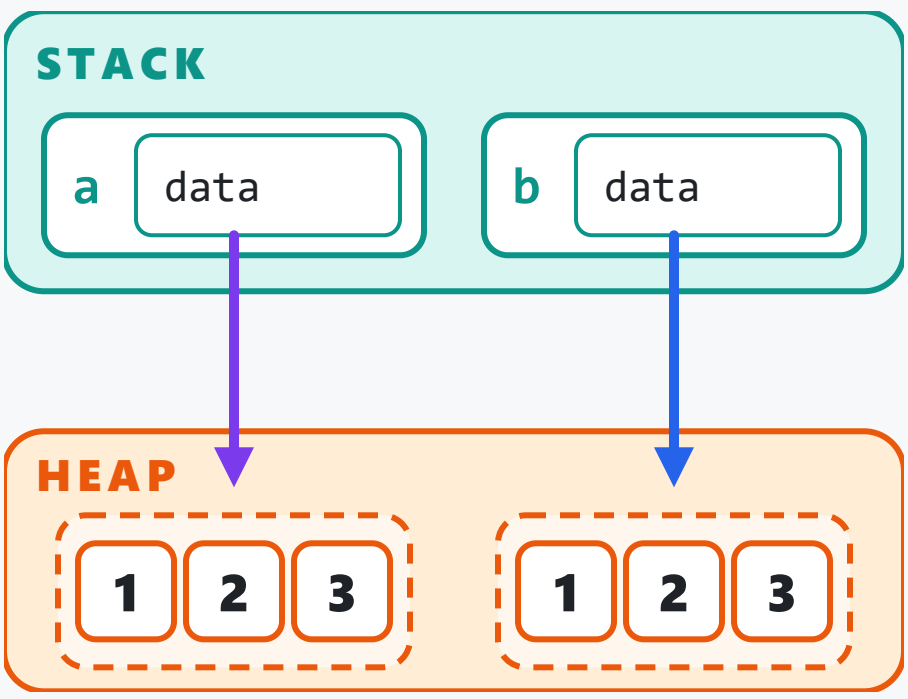
f(s);                    // 1
f(std::string("hi"));    // 2
```

The overload tells the function whether it may take the contents.

Why: copy or steal

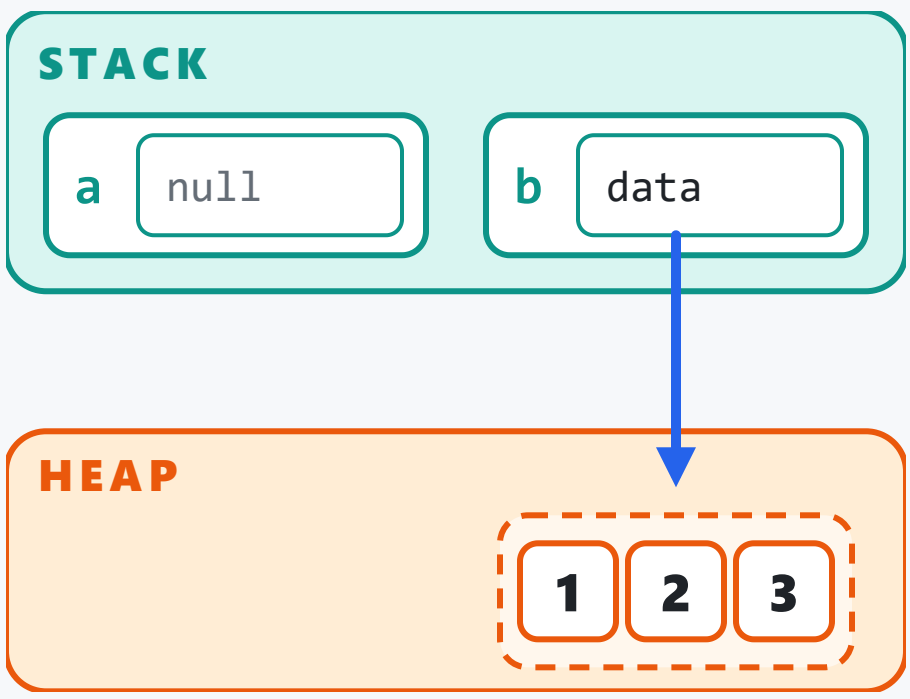
```
std::vector<int> b = a;                 // copy
std::vector<int> b = std::move(a);      // move
```

Copy



New block, new ints.

Move



Same block, new owner.

A move copies one pointer, not the elements.

`std::move` moves nothing

```
std::string a = "hello";
std::string b = std::move(a);
```

It is only a cast: **“treat `a` as an rvalue”**.

The move itself happens in the move constructor of `std::string`.

After it, `a` is moved-from.

- still valid, contents unspecified
- only destroy it or assign to it

Writing the move constructor

```
Numbers(Numbers&& o) noexcept
    : data(o.data) {
    o.data = nullptr;
}
```

Take the pointer. Leave the source empty.

The destructor stays safe: `delete[] nullptr` does nothing. Mark it `noexcept`, or `std::vector` will copy instead of move when it grows.

Do you write move operations or let the compiler?